

1. Introduction

These notes are meant to be read together with the lecture slides, but they are written in a more detailed style so that a student can study the material directly from the text. Our goal is to move gradually from the basic formulation of reinforcement learning to several modern ideas that have become important in large language model alignment, including RLHF, PPO, DPO, and GRPO. Throughout the notes, we will emphasize not only the final formulas, but also the intuition behind them, the role played by each quantity, and the connections between apparently different methods.

Reinforcement learning differs from standard supervised learning in a fundamental way. In supervised learning, we are given input-output pairs and we try to learn a mapping from inputs to desired outputs. In reinforcement learning, the learner is an agent interacting with an environment over time. The agent chooses an action, the environment evolves, and the agent receives a reward. The consequence of a decision may not be fully visible immediately. A good action may only reveal its benefit many steps later, and a bad action may create delayed harm. Because of this, reinforcement learning is inherently a problem of sequential decision making under uncertainty.

This sequential nature forces us to think beyond immediate rewards. A rational agent should care not just about what happens now, but about how the current action affects the entire future trajectory. This is why reinforcement learning introduces concepts such as return, value functions, state-action values, policies, and advantage functions. These concepts allow us to reason mathematically about long-term consequences.

2. Markov Decision Processes

The standard mathematical model for reinforcement learning is the Markov Decision Process, or MDP. An MDP is usually written as a tuple

$$(\mathcal{S}, \mathcal{A}, P, r, \gamma),$$

where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P(s' | s, a)$ is the transition distribution, $r(s, a)$ is the reward function, and $\gamma \in [0, 1)$ is the discount factor.

The state s_t summarizes what is currently relevant about the environment at time step t . The agent observes the state and selects an action a_t . The environment then produces the next state s_{t+1} according to the transition law $P(s_{t+1} | s_t, a_t)$ and gives the agent a reward r_t . The process then repeats. The discount factor γ controls how much the agent values the future. If γ is close to 0, the agent focuses mostly on immediate rewards. If γ is close to 1, the agent becomes far-sighted and cares strongly about future outcomes.

The key modeling assumption is the Markov property. This means that once the current state is known, the next state depends only on the current state and action, not on the full history. In other words, the present is assumed to contain all information needed to predict the future. This assumption is mathematically powerful because it allows us to define optimal decision rules based on the current state alone.

A policy $\pi(a | s)$ specifies how the agent behaves. It is a conditional distribution over actions given the current state. For a deterministic policy, the action is a fixed function of the state. For a stochastic policy, the agent samples an action from a distribution. In modern reinforcement

learning, especially in policy-gradient methods and RLHF, stochastic policies are extremely important because they enable exploration and give us differentiable objectives.

The central objective in reinforcement learning is to maximize the expected discounted return

$$J(\pi) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right].$$

This objective already reveals the main challenge of RL. The quality of an action cannot be judged solely by its immediate reward. A seemingly modest action may lead to highly rewarding future states, while a tempting immediate reward may send the agent into a poor region of the state space.

3. Return, Value Functions, and the Bellman View

To solve an RL problem, we need a way to quantify how good a state or action is under a policy. This leads to the notion of value functions. Let the return from time t be

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}.$$

The state-value function of a policy π is defined by

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid s_t = s].$$

This quantity tells us the expected long-term reward if we start in state s and then follow policy π . Similarly, the action-value function is defined by

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid s_t = s, a_t = a].$$

This quantity measures the quality of choosing action a in state s and then following policy π thereafter.

A third useful object is the advantage function

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s).$$

The advantage does not ask whether an action is good in an absolute sense. Instead, it asks whether the action is better or worse than what is typically expected in that state under the policy. This relative viewpoint is extremely important in policy-gradient methods. In fact, many modern algorithms update the policy based on whether an action performed better or worse than expected, rather than using raw rewards directly.

The Bellman equations express the recursive structure of these quantities. For the state value, we have

$$V^{\pi}(s) = \mathbb{E}_{a \sim \pi(\cdot|s)} [r(s, a) + \gamma V^{\pi}(s')].$$

For the action value, we have

$$Q^{\pi}(s, a) = \mathbb{E} [r(s, a) + \gamma \mathbb{E}_{a' \sim \pi(\cdot|s')} Q^{\pi}(s', a')].$$

These equations say that value can be decomposed into an immediate reward plus the discounted value of the future. This simple recursive principle is one of the central ideas of reinforcement learning.

If instead of evaluating a fixed policy we seek the best possible behavior, we obtain the optimal action-value function

$$Q^*(s, a) = \max_{\pi} Q^{\pi}(s, a),$$

which satisfies the Bellman optimality equation

$$Q^*(s, a) = \mathbb{E} \left[r(s, a) + \gamma \max_{a'} Q^*(s', a') \right].$$

This equation forms the foundation of Q-learning.

4. Q-Learning

Q-learning is one of the most influential algorithms in reinforcement learning. Its goal is to directly learn $Q^*(s, a)$, the optimal action-value function. Once Q^* is known, acting optimally becomes straightforward: at state s , choose the action that maximizes $Q^*(s, a)$.

The update rule of Q-learning is

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left(r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right),$$

where α is the learning rate. The quantity inside the parentheses is the temporal-difference error, often abbreviated as the TD error:

$$\delta = r + \gamma \max_{a'} Q(s', a') - Q(s, a).$$

This error measures how much the current estimate disagrees with a one-step bootstrap target.

Several important ideas are hidden inside this simple formula. First, Q-learning is off-policy. The update uses $\max_{a'} Q(s', a')$, which corresponds to the greedy action under the current Q-function, even if the behavior policy used to generate the data was exploratory. Second, Q-learning uses bootstrapping. Instead of waiting until the end of a trajectory to compute a full return, it updates using its own estimate of the future. This makes learning efficient, but it also introduces instability when function approximation is used.

It is worth pausing to understand what bootstrapping means. Suppose the agent moves from s to s' and receives reward r . If we somehow already knew the optimal value of s' , then a reasonable estimate of the value of (s, a) would be the immediate reward plus the discounted optimal future value. Q-learning simply replaces this ideal future value by the current estimate. It therefore improves itself using its own predictions.

5. From Tabular Q-Learning to Deep Q-Networks

In small environments with a finite number of states and actions, it is possible to store all Q-values explicitly in a table. But in many real problems, the state space is enormous or continuous. For example, if the input is an image, the state cannot be enumerated. This motivates the use of function approximation.

In Deep Q-Networks, or DQN, we replace the tabular function $Q(s, a)$ by a neural network $Q_\theta(s, a)$ with parameters θ . The training target is typically

$$y = r + \gamma \max_{a'} Q_{\theta^-}(s', a'),$$

where θ^- are the parameters of a target network. The loss function is

$$\mathcal{L}(\theta) = \mathbb{E} \left[\left(Q_\theta(s, a) - y \right)^2 \right].$$

At first glance, this looks like an ordinary regression problem, but it is much more delicate than supervised learning. The targets depend on the network itself, and successive samples are highly

correlated because they come from an evolving trajectory generated by the current policy. These two issues can lead to divergence or severe instability.

The original DQN paper introduced two ideas to address this. The first is experience replay. Instead of updating on consecutive transitions, the algorithm stores past transitions in a replay buffer and samples mini-batches uniformly at random. This breaks temporal correlations and makes the training distribution more stationary. The second is the target network. Rather than using the same network to generate both predictions and targets, DQN maintains a slowly updated copy Q_{θ^-} to compute targets. This stabilizes learning because the target does not move too rapidly.

Even with these improvements, DQN has important limitations. It is naturally suited to discrete action spaces, because of the maximization over actions. It can also suffer from overestimation bias due to the max operator. These issues led to many extensions, but for our purposes the most important conceptual point is this: DQN shows how value-based reinforcement learning can be combined with deep neural networks, making RL practical for high-dimensional inputs.

6. Policy Gradient Methods

Value-based methods learn a score for each action and then act by maximizing that score. Policy-gradient methods take a different perspective. Instead of learning values first and deriving a policy from them, they directly optimize a parameterized policy $\pi_{\theta}(a | s)$.

The central object is the expected return as a function of policy parameters, written as $J(\theta)$. The policy gradient theorem states that

$$\nabla_{\theta} J(\theta) = \mathbb{E} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) Q^{\pi}(s_t, a_t)].$$

This formula is extremely important. It says that the policy should be adjusted in the direction that increases the log-probability of actions that lead to high return. The factor $\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$ tells us how to change the policy parameters to make the chosen action more or less likely. The factor $Q^{\pi}(s_t, a_t)$ determines whether doing so is desirable.

In practice, using $Q^{\pi}(s_t, a_t)$ directly can lead to high variance. A standard trick is to subtract a baseline $b(s_t)$, which does not change the expectation of the gradient as long as the baseline does not depend on the action. Taking $b(s_t) = V^{\pi}(s_t)$ gives the advantage function:

$$\nabla_{\theta} J(\theta) = \mathbb{E} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) A^{\pi}(s_t, a_t)].$$

This form is especially intuitive. If an action performs better than expected, then its advantage is positive and its probability should increase. If it performs worse than expected, then its probability should decrease.

This interpretation becomes particularly useful when we later study PPO and RLHF. In those settings, the central policy update still follows this same principle: actions with positive advantage are encouraged, and actions with negative advantage are discouraged.

7. Advantage Estimation and GAE

To use policy gradients effectively, we need good estimates of the advantage. A poor advantage estimate can dramatically increase variance and destabilize training. One possibility is to use full Monte Carlo returns, which are unbiased but noisy. Another is to use one-step TD estimates, which are lower variance but biased. Generalized Advantage Estimation, or GAE, provides a continuum between these two extremes.

Define the one-step temporal-difference residual by

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t).$$

Then the GAE estimate of the advantage is

$$A_t^{\text{GAE}(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}.$$

The parameter $\lambda \in [0, 1]$ controls the bias-variance trade-off. When $\lambda = 0$, the estimate uses only the one-step TD residual. This has relatively low variance but more bias. When λ approaches 1, the estimate becomes closer to a Monte Carlo-style return, reducing bias but increasing variance.

A useful way to explain GAE intuitively is to say that it computes not simply whether the outcome was good, but whether it was better or worse than what the value function had predicted, while also aggregating such surprises over future time steps with an exponential discount. In policy-gradient algorithms such as PPO, the advantage is the signal that tells the policy how to move.

8. Proximal Policy Optimization

Proximal Policy Optimization, or PPO, is one of the most widely used policy-gradient algorithms because it achieves a good balance between simplicity and stability. The main idea of PPO is easy to state: we would like to improve the policy, but we do not want to change it too aggressively in a single update.

Suppose the old policy is π_{old} and the new policy is π_{θ} . PPO defines the ratio

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\text{old}}(a_t | s_t)}.$$

If this ratio is greater than 1, then the new policy assigns higher probability to the sampled action than the old policy did. If the ratio is less than 1, then the new policy assigns lower probability.

Without any special mechanism, a natural objective would be to maximize

$$\mathbb{E}[r_t(\theta) A_t].$$

This is essentially a policy-gradient surrogate objective. However, it can be unstable. If the update is too large, the policy may change drastically and the performance can collapse.

PPO addresses this by clipping the ratio:

$$\mathcal{L}^{\text{PPO}}(\theta) = \mathbb{E}[\min(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t)].$$

This objective is maximized. In practice, implementations often minimize its negative, but conceptually PPO is trying to maximize a clipped surrogate objective.

The clipping deserves a careful interpretation. Suppose $A_t > 0$, meaning the sampled action was better than expected. Then we want to increase its probability. But if the ratio $r_t(\theta)$ becomes too large, PPO stops rewarding further increase beyond the clipping threshold. Conversely, if $A_t < 0$, then we want to decrease the action probability, but PPO prevents the ratio from becoming too small too quickly. In this sense, PPO acts like a trust-region method: it allows policy improvement, but only within a controlled neighborhood of the old policy.

It is useful to emphasize to students that the advantage drives the direction of the update. Positive advantage says “make this action more likely,” and negative advantage says “make it less likely.” The ratio and clipping mechanism then determine how aggressively this should happen.

9. The Basic RLHF Pipeline

We now move from standard reinforcement learning to reinforcement learning from human feedback, or RLHF. The motivation is that in many language-generation tasks, we do not have a clean environment reward in the ordinary reinforcement learning sense. Instead, we want a model whose outputs align with human preferences. Helpfulness, harmlessness, honesty, stylistic appropriateness, and task usefulness are all properties that humans can judge, but they are hard to specify as hand-written reward functions.

A common RLHF pipeline has three stages. In Stage 1, we perform supervised fine-tuning, or SFT. A pretrained language model is fine-tuned on high-quality prompt-response pairs written by humans or curated by annotators. This produces a model that already behaves reasonably well.

In Stage 2, we train a reward model from human preference comparisons. Instead of asking annotators to assign absolute scores, which is difficult and inconsistent, we typically ask them to compare two candidate responses to the same prompt and indicate which one is better. From these pairwise comparisons, a reward model learns to assign a scalar score to a response.

In Stage 3, we optimize the policy, often using PPO, to maximize the learned reward while staying close to a reference model, usually the SFT policy. This final step is the “RL” part of RLHF.

This three-stage pipeline is conceptually elegant because each stage solves a different problem. SFT teaches the model how plausible answers should look. The reward model learns what humans prefer. PPO then improves the model using that learned notion of quality, while keeping the resulting policy from drifting too far away from the stable pretrained behavior.

10. Stage 2: Reward Modeling and the Bradley–Terry View

In Stage 2 of RLHF, the central question is how to learn a scalar reward from pairwise human preferences. Suppose that for a prompt x , annotators compare two responses $y^{(A)}$ and $y^{(B)}$, and indicate that $y^{(A)}$ is preferred over $y^{(B)}$. We would like to train a reward model $r_\phi(x, y)$ such that preferred responses receive higher scores.

The Bradley–Terry model gives a simple and natural probabilistic formulation. It assumes that each candidate response has a latent quality score and that the probability of one response beating another depends on the difference of these scores. Specifically,

$$P(y^{(A)} \succ y^{(B)} \mid x) = \frac{\exp(r_\phi(x, y^{(A)}))}{\exp(r_\phi(x, y^{(A)})) + \exp(r_\phi(x, y^{(B)}))}.$$

Equivalently,

$$P(y^{(A)} \succ y^{(B)} \mid x) = \sigma\left(r_\phi(x, y^{(A)}) - r_\phi(x, y^{(B)})\right),$$

where $\sigma(z) = 1/(1 + e^{-z})$ is the sigmoid function.

The most important conceptual point here is that preference is modeled as a relative judgment, not an absolute one. The reward model does not need to know the absolute meaning of a score such as 7.3 or 12.1. Only differences matter. If one response gets a larger reward than another, it should be more likely to be preferred.

This leads to the standard reward-model training loss

$$\mathcal{L}_{\text{RM}}(\phi) = -\mathbb{E} \left[\log \sigma \left(r_\phi(x, y_w) - r_\phi(x, y_l) \right) \right],$$

where y_w denotes the winning response and y_l the losing response. This is essentially a pairwise logistic regression objective.

There is also a useful interpretation of why the model depends only on reward differences. Since the preference task only asks which response is better, adding the same constant to all rewards for a given prompt changes nothing. Thus, the reward is only identifiable up to an additive shift. This is one reason why it is best thought of as a latent utility rather than a calibrated score.

11. Stage 3: PPO Fine-Tuning in RLHF

In Stage 3, we use reinforcement learning to optimize the language model with respect to the reward model. It is important to distinguish the data used here from the preference data used in Stage 2. Stage 2 uses human-labeled comparison data of the form (x, y_w, y_l) . Stage 3, by contrast, uses on-policy samples generated by the current language model. For a prompt x , we sample a response $y \sim \pi_\theta(\cdot | x)$, score it using the reward model, and then update the policy.

Thus, Stage 2 is fundamentally offline supervised learning on a fixed human-labeled dataset, whereas Stage 3 is on-policy policy optimization. The reward model learned in Stage 2 is reused as an evaluator during Stage 3, but the actual training samples are generated by the evolving policy.

A very important ingredient in RLHF is the drift penalty, typically implemented as a KL regularizer that keeps the optimized policy close to a reference policy. If π_{ref} denotes the reference model, usually the SFT policy, then the RLHF objective can be written as

$$\max_{\pi} \mathbb{E}_{y \sim \pi(\cdot | x)} [r(x, y)] - \beta \text{KL}(\pi(\cdot | x) \| \pi_{\text{ref}}(\cdot | x)).$$

The KL term is sometimes called a drift penalty because it penalizes the new policy for drifting too far away from the reference model.

This penalty is not a decorative extra term. It is crucial. Without it, the policy may exploit imperfections of the reward model and produce unnatural outputs that achieve high reward only because the reward model is flawed. This phenomenon is often called reward hacking. The KL penalty also stabilizes training and preserves fluency, coherence, and general world knowledge inherited from the pretrained and SFT models.

12. GAE, the Value Model, and Why Advantage Feeds into the Policy Update

When PPO is used in RLHF, the overall structure resembles standard policy-gradient reinforcement learning. A value model estimates expected future return, a reward signal is computed from the reward model and regularization terms, and an advantage estimate is formed. That advantage then drives the policy update.

Students often ask what GAE means in diagrams of PPO-based RLHF systems. GAE stands for Generalized Advantage Estimation. It is the method used to compute a low-variance estimate of the advantage A_t . The value model predicts the expected return, while the realized reward and future values define a TD residual. GAE aggregates these residuals over time. The resulting quantity tells us how much better or worse a sampled action turned out to be relative to expectation.

This immediately explains why the advantage is sent to the policy model. The policy should not be updated according to raw reward alone, but according to whether the observed action

was better or worse than expected under the current model. If the advantage is positive, the action was surprisingly good and should become more likely. If the advantage is negative, the action was disappointing and should become less likely. In this sense, the advantage is the error signal that teaches the policy how to move.

13. The KL-Constrained RLHF Objective and Its Closed-Form Optimum

One of the most important theoretical observations behind modern alignment methods is that the KL-regularized RLHF objective has a closed-form optimal policy. Let us derive this carefully, because the derivation is both elegant and useful.

Fix a prompt x . Consider the optimization problem

$$\max_{\pi(\cdot|x)} \mathbb{E}_{y \sim \pi(\cdot|x)}[r(x, y)] - \beta \text{KL}(\pi(\cdot | x) \| \pi_{\text{ref}}(\cdot | x)).$$

Expanding the KL divergence gives

$$\text{KL}(\pi \| \pi_{\text{ref}}) = \sum_y \pi(y | x) \log \frac{\pi(y | x)}{\pi_{\text{ref}}(y | x)}.$$

Hence the objective becomes

$$J(\pi) = \sum_y \pi(y | x) r(x, y) - \beta \sum_y \pi(y | x) \log \frac{\pi(y | x)}{\pi_{\text{ref}}(y | x)}.$$

We must maximize this over all distributions $\pi(\cdot | x)$ satisfying

$$\sum_y \pi(y | x) = 1, \quad \pi(y | x) \geq 0.$$

Introduce a Lagrange multiplier λ for the normalization constraint. The Lagrangian is

$$\mathcal{J}(\pi, \lambda) = \sum_y \pi(y | x) r(x, y) - \beta \sum_y \pi(y | x) \log \frac{\pi(y | x)}{\pi_{\text{ref}}(y | x)} + \lambda \left(\sum_y \pi(y | x) - 1 \right).$$

Now differentiate with respect to $\pi(y | x)$. Using the identity

$$\frac{d}{dz} (z \log z) = \log z + 1,$$

we obtain

$$\frac{\partial \mathcal{J}}{\partial \pi(y | x)} = r(x, y) - \beta \left(\log \frac{\pi(y | x)}{\pi_{\text{ref}}(y | x)} + 1 \right) + \lambda.$$

Setting this derivative to zero yields

$$r(x, y) - \beta \left(\log \frac{\pi(y | x)}{\pi_{\text{ref}}(y | x)} + 1 \right) + \lambda = 0.$$

Rearranging,

$$\beta \log \frac{\pi(y | x)}{\pi_{\text{ref}}(y | x)} = r(x, y) + \lambda - \beta.$$

Exponentiating both sides gives

$$\pi(y | x) = \pi_{\text{ref}}(y | x) \exp \left(\frac{r(x, y)}{\beta} \right) \exp \left(\frac{\lambda - \beta}{\beta} \right).$$

The final exponential factor does not depend on y , so it is simply a normalization constant. Therefore,

$$\pi^*(y | x) \propto \pi_{\text{ref}}(y | x) \exp \left(\frac{r(x, y)}{\beta} \right).$$

Normalizing over all responses gives the closed-form optimum

$$\pi^*(y \mid x) = \frac{\pi_{\text{ref}}(y \mid x) \exp(r(x, y)/\beta)}{\sum_{y'} \pi_{\text{ref}}(y' \mid x) \exp(r(x, y')/\beta)}.$$

This formula is extremely revealing. It says that the optimal aligned policy is obtained by exponentially reweighting the reference policy according to reward. In other words, the reward does not define a policy from scratch. Rather, it tilts the reference policy toward preferred responses. The scalar β controls the strength of this tilting. A large β keeps the policy close to the reference model, while a small β allows the reward to dominate more strongly.

14. Direct Preference Optimization

The derivation above is the key entry point to DPO, or Direct Preference Optimization. DPO starts from the observation that if the optimal policy under the KL-regularized objective has the form

$$\pi^*(y \mid x) \propto \pi_{\text{ref}}(y \mid x) \exp(r(x, y)/\beta),$$

then we can invert this relation to express reward differences in terms of policy log-ratio differences.

Taking logarithms of the closed-form expression gives

$$r(x, y) = \beta \log \frac{\pi^*(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x),$$

where $Z(x)$ is the partition function associated with the normalization constant. When comparing two responses y_w and y_l for the same prompt, the $\log Z(x)$ term cancels. Therefore,

$$r(x, y_w) - r(x, y_l) = \beta \log \frac{\pi^*(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi^*(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}.$$

If we now plug this reward difference into the Bradley–Terry preference model, we obtain

$$P(y_w \succ y_l \mid x) = \sigma \left(\beta \log \frac{\pi_{\theta}(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_{\theta}(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} \right).$$

This leads directly to the DPO loss

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_{\theta}(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} \right) \right].$$

Thus DPO eliminates the explicit reward model and the explicit PPO stage. Instead of first learning a reward and then optimizing the policy with RL, DPO directly optimizes the policy on pairwise preference data. This is one reason it often appears mathematically cleaner or more elegant than PPO-style RLHF.

15. How to Interpret the DPO Gradient

The DPO gradient has a particularly nice interpretation, and this is worth understanding because it clarifies why DPO works. Define

$$\Delta = \beta \left[\log \frac{\pi_{\theta}(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \log \frac{\pi_{\theta}(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} \right].$$

Then the DPO loss for a single pair is simply

$$\mathcal{L}_{\text{DPO}} = -\log \sigma(\Delta).$$

Differentiating with respect to Δ gives

$$\frac{\partial \mathcal{L}_{\text{DPO}}}{\partial \Delta} = -(1 - \sigma(\Delta)).$$

By the chain rule,

$$\nabla_{\theta} \mathcal{L}_{\text{DPO}} = -\beta(1 - \sigma(\Delta)) [\nabla_{\theta} \log \pi_{\theta}(y_w | x) - \nabla_{\theta} \log \pi_{\theta}(y_l | x)].$$

This formula is extremely intuitive. The gradient tries to increase the probability of the preferred response and decrease the probability of the dispreferred response. Moreover, the multiplicative factor $(1 - \sigma(\Delta))$ acts as an adaptive weight. If the model already strongly prefers the winner, then Δ is large, $\sigma(\Delta)$ is close to 1, and the gradient is small. If the model ranks the pair incorrectly, then Δ is small or negative, and the gradient becomes strong. In other words, DPO automatically focuses learning on hard or mis-ranked pairs.

This is one of the elegant aspects of DPO: it behaves like a pairwise policy-gradient update without needing a reward model, a value function, or advantage estimation.

16. Why DPO Often Feels More Elegant than GRPO

It is natural to feel that DPO is more elegant than GRPO, because mathematically it often is. DPO turns the alignment problem into a supervised-style objective on preference pairs. It uses the closed-form structure of the KL-regularized optimum and removes several moving parts: no explicit reward model at training time, no critic, no GAE, and no PPO clipping. The resulting pipeline is conceptually compact.

However, this elegance comes with assumptions. DPO is fundamentally an offline preference-learning method. It works with fixed pairs of preferred and dispreferred responses. GRPO, by contrast, remains within the reinforcement-learning framework. It can optimize explicit scalar rewards on on-policy samples, which may allow greater flexibility in settings where richer reward signals are available. Therefore, one may say that DPO removes much of the pain of RL, but it also removes some of RL’s power.

17. Group Relative Policy Optimization

GRPO, or Group Relative Policy Optimization, can be understood as a simplification of PPO designed to avoid explicit value-function learning. In PPO, we need a value model in order to estimate advantages. This critic can be difficult to train well, and poor value estimates can destabilize learning. GRPO tries to remove this dependency.

The idea is straightforward. For a given prompt x , instead of generating a single response, we sample a group of responses

$$y_1, y_2, \dots, y_K \sim \pi_{\theta}(\cdot | x).$$

Each response is scored by a reward function, giving rewards $r_1, r_2, \dots, r_K$. Rather than estimating a separate value baseline, GRPO uses the group mean reward as a baseline. The relative advantage of sample i is

$$A_i = r_i - \frac{1}{K} \sum_{j=1}^K r_j.$$

This is conceptually simple but important. A response is judged relative to other responses generated for the same prompt under the same policy. If its reward is above the group average, it receives positive advantage and is encouraged. If its reward is below average, it receives negative advantage and is discouraged.

One can then plug this relative advantage into a PPO-style clipped objective. Compared with PPO, the key difference is not in the outer optimization structure but in how the advantage is constructed. PPO estimates advantage using a value model and often GAE; GRPO estimates it by relative comparison within a sampled group.

This makes GRPO attractive in language-model alignment settings because it avoids critic training while preserving an explicit reinforcement-learning flavor. At a high level, PPO uses value-based advantage estimation, GRPO uses group-relative advantage estimation, and DPO bypasses explicit RL entirely.

18. A Unified View of PPO, DPO, and GRPO

Although PPO, DPO, and GRPO look rather different operationally, they can be understood within a common conceptual framework. The shared starting point is the KL-regularized objective

$$\max_{\pi} \mathbb{E}_{y \sim \pi} [r(x, y)] - \beta \text{KL}(\pi \| \pi_{\text{ref}}).$$

This objective reflects the central alignment tension. On the one hand, we want to move toward responses that humans prefer. On the other hand, we do not want to move so far away from the reference model that fluency, calibration, or stability collapse.

PPO treats this as a genuine reinforcement-learning problem. It uses on-policy samples, an explicit reward model, a value model, and advantage estimation. GRPO keeps the reinforcement-learning structure but simplifies advantage estimation by comparing samples within a group. DPO takes a different route: it exploits the closed-form optimum induced by the KL-regularized objective and turns the problem into a direct preference optimization loss.

This unified view helps explain both the similarities and differences among these methods. All of them can be interpreted as ways of reweighting a strong reference model toward human preferences. They differ mainly in whether reward is explicit or implicit, whether training is on-policy or offline, and whether reinforcement learning is retained or removed.

19. Concluding Remarks

The historical progression from classical RL to modern alignment methods is not a sequence of unrelated tricks. It is a chain of ideas connected by a common mathematical structure. Value functions and Bellman equations teach us how to think about long-term consequences. Q-learning shows how to approximate optimal decision values. Policy gradients shift attention from values to directly learnable stochastic policies. PPO adds a mechanism for conservative policy updates. RLHF uses these tools to optimize large language models with respect to learned human preference signals. DPO and GRPO then revisit the RLHF pipeline and simplify or reinterpret it in different ways.

When studying these methods, it is useful not to memorize isolated formulas. Instead, try to identify the role of each object. Ask what is playing the role of reward, what acts as the baseline, where the policy is being regularized, and whether learning is happening through explicit reinforcement learning or through a supervised surrogate derived from the same objective. Once these questions become natural, the field becomes much easier to navigate.